

Campus Navigator

Project Overview & Demo Reference

Course project — data structures team build

Demo date: May 6, 2026

Team

Ian Dudley	Graph (adjacency list, BFS pathfinding)
Jack Meehan	NameHashTable (separate-chaining hash table)
Hailey Pieper	SortedIndex (binary search tree)
Tomi Babayemi	FileIO (load & save dataset)
Jody Spikes	Campus integration hub & main menu

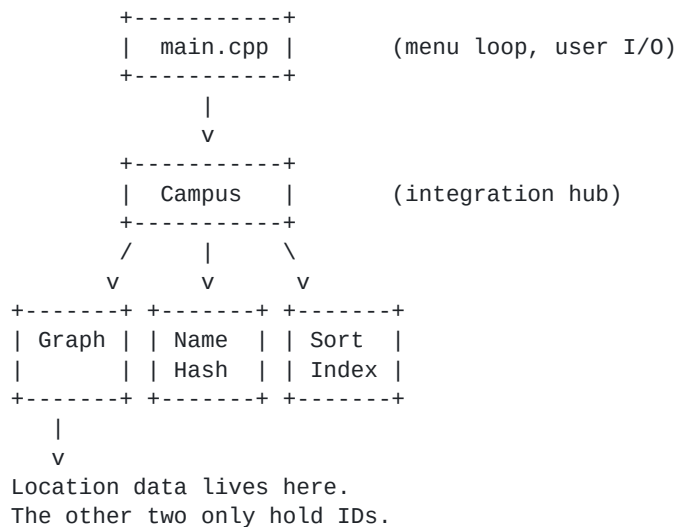
1. What the system does

Campus Navigator is an interactive command-line application for managing a university campus map. A user can add, remove, update, and search locations; list locations by category or alphabetically; and find a route between any two locations on the map. The dataset loads from two pipe-delimited text files at startup and can be edited live through a numbered menu.

The project's purpose is to demonstrate three complementary data structures working together to support different kinds of queries efficiently. No single structure can do all three jobs well, so the system uses each one for what it's best at and keeps them in sync through a central integration class.

2. Architecture at a glance

The Graph is the single source of truth for Location data. The hash table and the binary search tree store only LocationIDs that refer back into the Graph. The Campus class is the only thing that mutates state; it is responsible for keeping all three structures consistent.



Why three structures? Each query type has a different ideal data structure. Looking up by name is $O(1)$ average in the hash table. Listing alphabetically is $O(n)$ in-order traversal of the BST. Finding a route is BFS over the graph's adjacency list. Trying to do all three with one structure would force at least one query to go quadratic.

3. Component breakdown

Location.h / Location.cpp — Shared types

Owner: shared (locked in Week 1)

Defines the foundation types every other module depends on: the **LocationID** alias (an int with -1 sentinel **INVALID_ID**), the **Category** enum, and the **Location** plain-data struct. Also supplies free functions **categoryToString** and **stringToCategory** (case-insensitive). This file was deliberately frozen early so changes here don't ripple through the rest of the project.

Graph.h / Graph.cpp — The campus map

Owner: Ian Dudley

Stores Location objects and the connections between them. Locations live in a **vector<Location>** indexed by ID; a parallel **vector<bool>** tracks which slots are still active so removing a location does not invalidate IDs held by other modules. Edges are stored in an adjacency list (**vector<vector<Edge>>**), and every edge is added in both directions because the graph is undirected.

Key public API:

```
LocationID addLocation(name, category, description);
bool      removeLocation(LocationID id);
bool      updateLocation(id, name, category, description);
const Location* getLocation(LocationID id) const;
vector<LocationID> getAllIDs() const;

bool addEdge(LocationID from, LocationID to, double distance);
bool removeEdge(LocationID from, LocationID to);
vector<pair<LocationID, double>> getNeighbors(LocationID id) const;

vector<LocationID> bfsPath(from, to) const;          // shortest hops
vector<LocationID> dijkstraPath(from, to) const;     // extension
```

BFS implementation note: the search tracks visited nodes and parent pointers, then walks the parents back from the destination to reconstruct the path. Inactive (removed) nodes are skipped during traversal as a safety net.

NameHashTable.h / NameHashTable.cpp — Fast name lookup

Owner: Jack Meehan

A separate-chaining hash table mapping location **name** -> **LocationID**. Buckets are linked lists of **Node** objects. The capacity is always a prime number (**nextPrime**) and the table rehashes automatically when the load factor exceeds 0.75. Tracks collision and rehash counts as instrumentation that can be reported in the writeup.

Key public API:

```
bool insert(const string& name, const int& id);
bool remove(const string& name);
int find(const string& name) const; // returns -1 on miss
bool contains(const string& name) const;
size_t size(), tableSize();
double loadFactor();
size_t collisionCount(), rehashCount();
```

Why a hash table? The menu's "search by name" feature, plus every internal name-to-ID resolution (used by add-edge, route, remove-location, update-location), needs $O(1)$ average lookup. Walking the BST every time would be $O(\log n)$ and would dominate the route command's runtime.

SortedIndex.h / SortedIndex.cpp — Alphabetical traversal

Owner: Hailey Pieper

A binary search tree keyed by location name, storing LocationIDs at each node. Used to produce alphabetically sorted listings (option 1, and option 3 within a category). Remove uses the standard in-order-successor replacement strategy. An optional **findByPrefix** traversal is provided for autocomplete-style queries.

Key public API:

```
void insert(const string& name, LocationID id);
bool remove(const string& name);
vector<LocationID> inOrder() const; // sorted A-Z
vector<LocationID> findByPrefix(const string&) const; // bonus
```

Why a BST? Hash tables don't preserve order; getting alphabetical output from one would require sorting on every query. The BST gives us sorted iteration in a single in-order traversal.

FileIO.h / FileIO.cpp — Persistence

Owner: Tomi Babayemi

Pure functions in the **FileIO** namespace. **loadCampus** reads two pipe-delimited files (locations and edges), parses each line, and populates the Graph, hash table, and sorted index together. **saveCampus** writes the current graph state back out, taking care to write each undirected edge only once (skipping duplicates by emitting only when *from* < *to*).

File format:

```
# locations.txt
# id|name|category|description
0|Thompson Hall|Academic|Main CS building
```

```
1|Founders Dining|Dining|Central dining hall
```

```
# edges.txt
```

```
# from_id|to_id|distance
```

```
0|1|150.5
```

Pipes were chosen as the delimiter because building names commonly contain commas ("Jones Hall, East Wing") and spaces. Comment lines starting with # and the optional header line are skipped during parsing.

Campus.h / Campus.cpp — Integration hub

Owner: Jody Spikes

The single class **main.cpp** talks to. Holds one of each underlying structure as a member and exposes a name-based API to callers (so callers never have to know about LocationIDs unless they want to). The golden rule for this class: every mutation must update the graph, hash table, and sorted index together, in an order that doesn't leave the system half-changed.

Key public API:

```
bool      load(locFile, edgeFile);
bool      save(locFile, edgeFile) const;

LocationID addLocation(name, category, description);
bool      removeLocation(name);
bool      updateLocation(name, newName, newCat, newDesc);
bool      addConnection(fromName, toName, distance);
bool      removeConnection(fromName, toName);

LocationID findByName(name) const;
vector<LocationID> listAll() const;
vector<LocationID> listByCategory(Category c) const;
vector<LocationID> route(fromName, toName) const;
```

Mutation order matters. For removes, look up the ID via the hash table *before* removing the name from any structure. For updates, the hash and tree are rebuilt only when the name actually changes. The **route** method is a thin wrapper that resolves both names to IDs and delegates to

Graph::bfsPath.

main.cpp — Menu loop

Owner: Jody Spikes

Entry point. Prints an ASCII-art logo, attempts to load the default dataset, and enters an infinite menu loop. Each menu choice maps directly to one Campus method. Three small helpers keep the cases compact: **readLine** for any string field, **promptCategory** for parsing category names, and **printLocation** for consistent location display.

4. Menu walkthrough

Each menu option calls one Campus method. The table below summarizes which structures each option touches. "Read" means the option queries that structure; "Write" means it mutates it.

#	Action	Graph	Hash	BST
1	List all (A-Z)	Read	-	Read
2	Search by name	Read	Read	-
3	List by category	Read	-	Read
4	Route between two	Read	Read	-
5	Add location	Write	Write	Write
6	Remove location	Write	Write	Write
7	Add connection	Write	Read	-
8	Remove connection	Write	Read	-
9	Update location	Write	Write*	Write*
0	Quit	-	-	-

* Update only writes to the hash and tree if the name actually changes. If the new name equals the old name, only the graph is updated.

5. Suggested demo flow

A 4-5 minute demo can comfortably hit every component. One possible script:

1. Run the binary; the dataset loads from **data/locations.txt** and **data/edges.txt** — this exercises FileIO, Graph, NameHashTable, and SortedIndex all at once.
2. Choose option **1** to list everything alphabetically. Point out that the order comes from the BST, not the graph.
3. Choose option **2** and search for a known building. Point out that the lookup is $O(1)$ average through the hash table.
4. Choose option **3** and filter by a category (e.g. Academic). This combines BST traversal with a category check.
5. Choose option **4** to find a route between two distant locations. Talk about BFS and how the path is reconstructed.
6. Choose option **5** to add a new location, then option **7** to connect it. Repeat option **1** to show all three structures stayed in sync.
7. Choose option **6** to remove a location. Point out that the ID is marked inactive rather than shifted, so other IDs stay valid — and that all touching edges are cleaned up.
8. Wrap up: mention the optional Dijkstra extension and any extras.

6. Building and running

The Makefile compiles every **.cpp** in **src/** with C++17 and **-Wall -Wextra**. Two targets:

```
make          # builds ./campus_nav
make run      # builds and runs
make clean    # removes objects and the binary
```

The binary expects **data/locations.txt** and **data/edges.txt** in the current working directory. If either is missing, the program still launches with an empty dataset and the user can populate it through the menu.

7. Known limitations & future work

Honest list of things that are good to acknowledge in the writeup:

- **Duplicate names.** Adding two locations with the same name leaves the original orphaned in the graph. Mitigated in Campus by checking `nameIndex_.find(name) != INVALID_ID` before each add.
- **Dijkstra not yet implemented.** The signature is in place but returns empty. BFS gives shortest hop count; for shortest weighted distance, Dijkstra is the natural extension and a clear extra-credit target.
- **BST is not self-balancing.** Worst case (sorted input) gives $O(n)$ operations. Upgrading to AVL would guarantee $O(\log n)$ and is worth writeup points.
- **Cross-platform line endings.** The file loader assumes Unix-style line endings; files saved on Windows can leave a stray `\r` in the last field of each line.
- **Save format.** `saveCampus` writes data lines without the `id|name|category|description` header. The loader tolerates this, but the round-trip output looks slightly different from the example.

8. Closing note

The project's strongest selling point is the clean separation of concerns. Each teammate owned a self-contained module with a small, well-defined public interface, and the Campus class stitched them together without leaking any of their internals. Anything that needs to change in one structure (say, swapping the BST for an AVL tree) can be done without touching the other four files.